



Props



Learning Objectives

- Explain that the difference between **React props** and **React state** is that **props** is used for state that doesn't need to change, while **state** is used for state that might change
- Explain that parent components can share state via props with child components
- Explain that when sharing data via **props**, a child component receives it as a property of the **props object** declared in the child component function argument
- Explain that parent components can share any data type via props including functions
- Model the state a react application needs using a component tree annotated with where state should be and whether it should be kept as React state or React props
- Build two react components where the parent component shares state via props including strings, integers, arrays and functions



Intro

We've now learned about **Components**, **JSX** and **State**

All those are really important concepts, and today we're looking at another core concept in React: **props**

Q: What do you think **props** is short for?



Seeing data

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <h1 className='title'>Welcome to React, {name}</h1>
      <p>Things are about to get fun!</p>
    </div>
  )
}
```

Q: Can the **h1** “see” the App’s **name** state?



Seeing data

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <Title />
      <p>Things are about to get fun!</p>
    </div>
  )
}
```

```
// Title.js
function Title () {
  return (
    <h1 className='title'>
      Welcome to React, {name}!
    </h1>
  )
}

export default Title
```

Let's extract the **h1** into a **Title** component instead

Q: Now can the **h1** see the **name**? Why? How could we fix it?



Seeing data

```
// App.js
function App () {
  return (
    <div className='app'>
      <Title />
      <p>Things are about to get fun!</p>
    </div>
  )
}
```

```
// Title.js
function Title () {
  const [name, setName] = useState('Booleans')
  return (
    <h1 className='title'>
      Welcome to React, {name}!
    </h1>
  )
}

export default Title
```

One way to fix it would be to **move the state** down to the **Title** component, since it's the only component that needs it



Seeing data

```
// App.js
function App () {
  return (
    <div className='app'>
      <Title />
      <p>Things are about to get fun, {name}!</p>
    </div>
  )
}
```

```
// Title.js
function Title () {
  const [name, setName] = useState('Booleans')
  return (
    <h1 className='title'>
      Welcome to React, {name}!
    </h1>
  )
}

export default Title
```

Q: But what if our `<p>` tag also needed it? Can that p element see the name now?



Props

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <Title name={name} />
      <p>Things are about to get fun, {name}!</p>
    </div>
  )
}
```

```
// Title.js
function Title (props) {
  return (
    <h1 className='title'>
      Welcome to React, {props.name}!
    </h1>
  )
}

export default Title
```

Our state can't be in two places at once! Luckily, React has a very neat solution for this, called **props**

Props is a mechanism that allows us to take **any data** from one component, and **share it** with a **child** component



Props

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <Title name={name} />
      <p>Things are about to get fun, {name}!</p>
    </div>
  )
}
```

```
// Title.js
function Title (props) {
  return (
    <h1 className='title'>
      Welcome to React, {props.name}!
    </h1>
  )
}

export default Title
```

Passing data to a component looks very similar to adding an attribute to an element

Once we pass it, our component receives it as a property of this **props object** we get as an argument



Prop names

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <Title name={name} />
      <p>Things are about to get fun, {name}!</p>
    </div>
  )
}
```

```
// Title.js
function Title (props) {
  return (
    <h1 className='title'>
      Welcome to React, {props.name}!
    </h1>
  )
}

export default Title
```

We can give our props **any arbitrary name**, it doesn't have to match the name of the thing we are passing



Prop names

```
// App.js
function App () {
  const [name, setName] = useState('Booleans')
  return (
    <div className='app'>
      <Title potato={name} />
      <p>Things are about to get fun, {name}!</p>
    </div>
  )
}
```

```
// Title.js
function Title (props) {
  return (
    <h1 className='title'>
      Welcome to React, {props.potato}!
    </h1>
  )
}

export default Title
```

For example, here, we renamed our prop to **potato**, and we need to change the code in our **receiving component** to match



Time for some live coding

Let's see this in action!



Time for some practice

<https://github.com/boolean-uk/react-props-practice-1>

- **Separate into Title, NameLength, and NameReversed components**
- **Make it so all of them still have access to 'name'**



Cleaning up

Remember our todo app?

Now that we know about props, we can start doing some serious **cleaning up**!

OPTIONS

Show completed ☒

ADD ITEM

Add

TODO

☐ Go shopping 🗑

☐ Work out 🗑

COMPLETED

☒ See the doctor 🗑



Breaking it down

Knowing that React allows us to separate things neatly into **components**, we can go through our app and spot the **different parts** that make it up

Q: Looking at the screenshot, what components do you see?

OPTIONS

Show completed ☒

ADD ITEM

Add

TODO

☐ Go shopping 

☐ Work out 

COMPLETED

☒ See the doctor 



Breaking it down

Here's what we've got!

`<Options />`

`<AddItem />`

`<TodoList />`

`<CompletedTodoList />`

`<App />`

OPTIONS

Show completed ☒

ADD ITEM

 Add

TODO

☐ Go shopping 

☐ Work out 

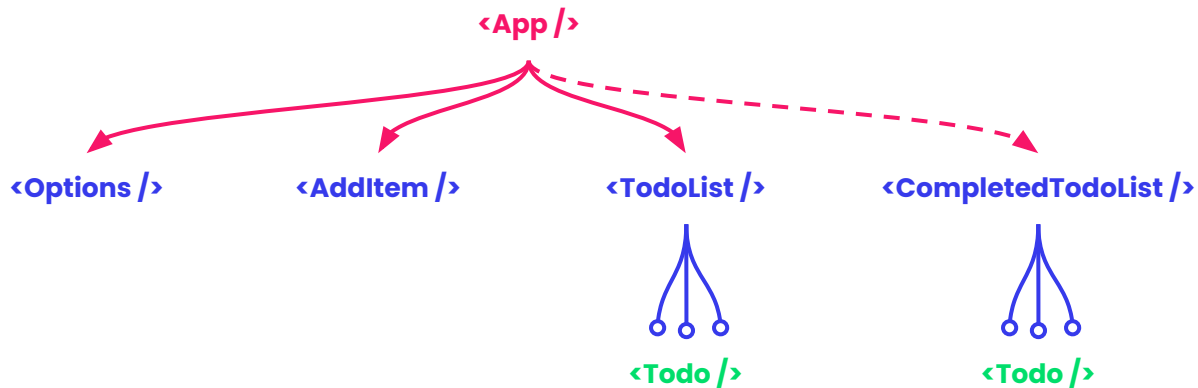
COMPLETED

☒ See the doctor 

`<Todo />`



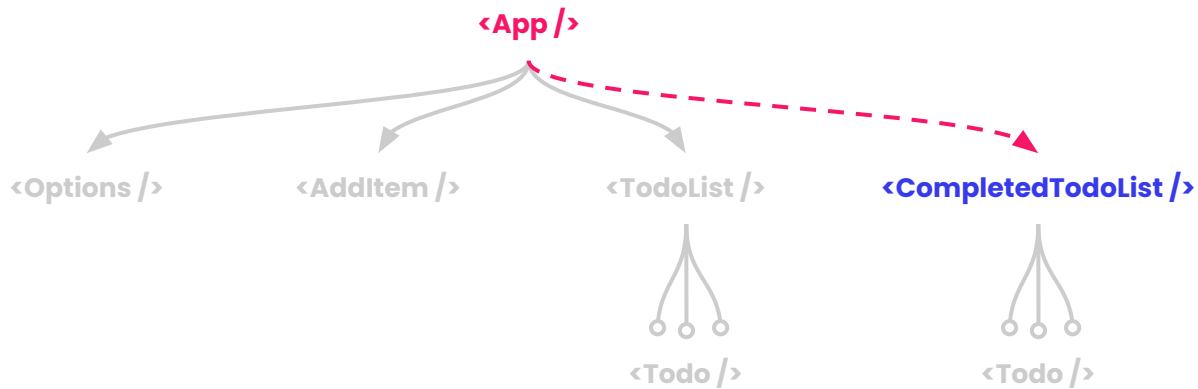
Component tree



It is useful to start thinking about your app as a **tree of components** with **parent – child relationships**



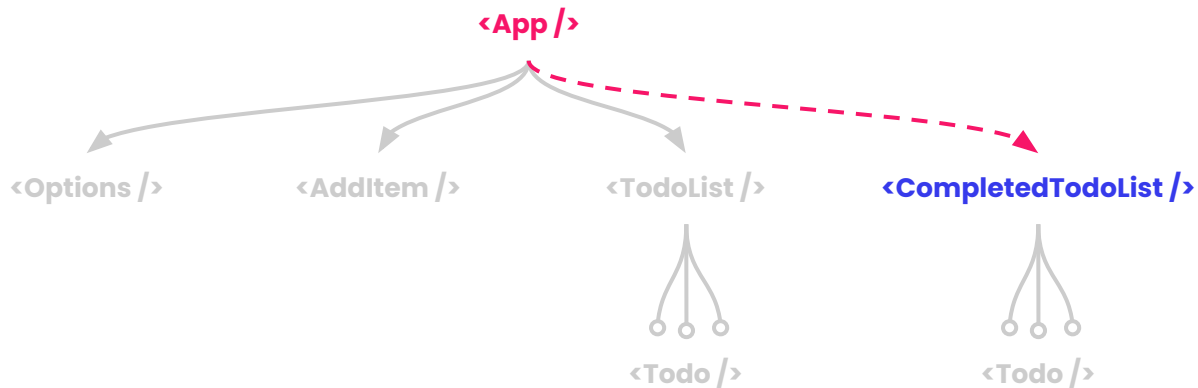
Component tree



Q: What do you think the **dotted line** is signalling here?



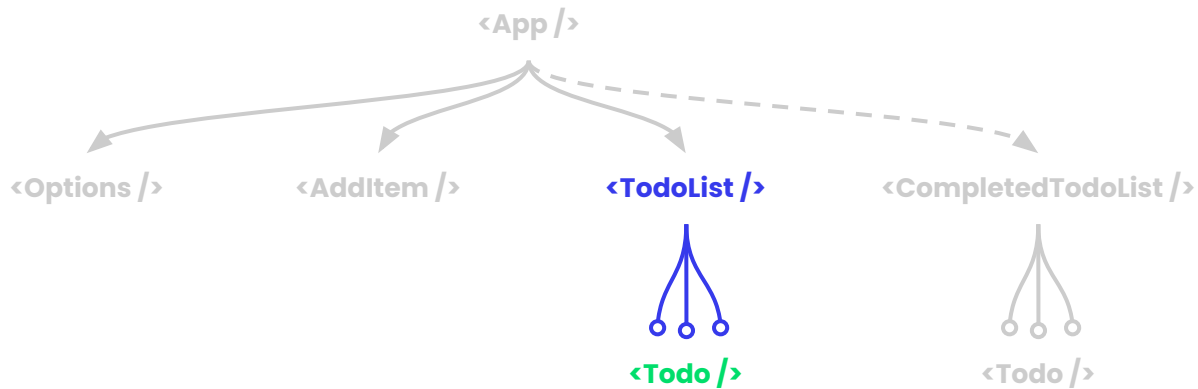
Component tree



A: The line illustrates that this component is being **conditionally rendered**, so it might not always show up on the page



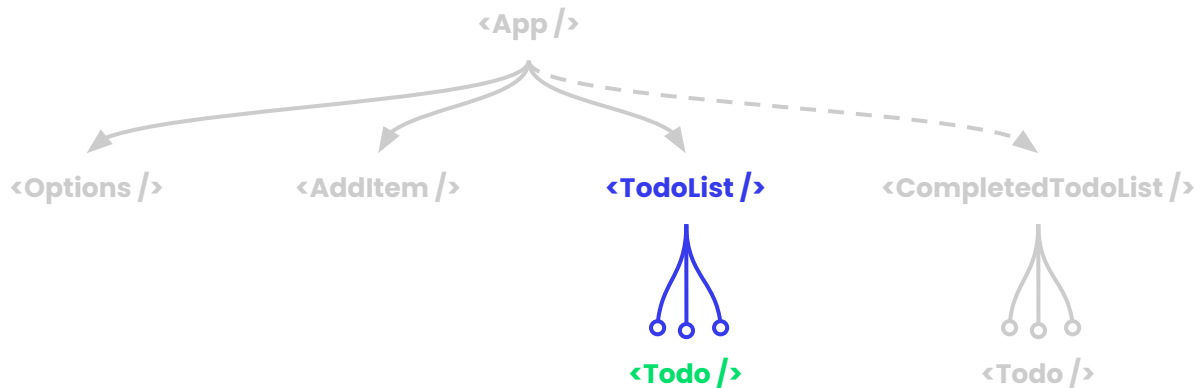
Component tree



Q: And what about these funky lines?



Component tree



A: They signal that this component is rendering **multiple todos**, we don't know exactly how many. Could be **0**, **1** or **10000**.



Time for some live coding

Now, it's time to clean up our Todo App together!

Here's a finished version



Passing functions as a prop

```
const [todos, setTodos] = useState(initialTodos)
```

```
const removeTodo = (target) => { setTodos(todos => todos.filter(todo => todo !== target)) }
```

```
<TodoList todos={todos} removeTodo={removeTodo} />
```

Remember, we can pass **any JS type** as a prop, which means we can certainly **pass down functions**, too!

Q: But what's the purpose of passing down a function here?



Passing functions as a prop

```
const [todos, setTodos] = useState(initialTodos)
```

```
const removeTodo = (target) => { setTodos(todos => todos.filter(todo => todo !== target)) }
```

```
<TodoList todos={todos} removeTodo={removeTodo} />
```

A: Our **TodoList** contains our **Todo** component, which needs to call this function in order to **remove itself** from the app's state

Q: Couldn't we just create our `removeTodo` function inside the `Todo` component? Why?



Passing functions as a prop

```
const [todos, setTodos] = useState(initialTodos)
```

```
const removeTodo = (target) => { setTodos(todos => todos.filter(todo => todo !== target)) }
```

```
<TodoList todos={todos} removeTodo={removeTodo} />
```

A: removeTodo needs access to **todos** and **setTodos** in order to work, so it **needs** to be written where they are



Key prop

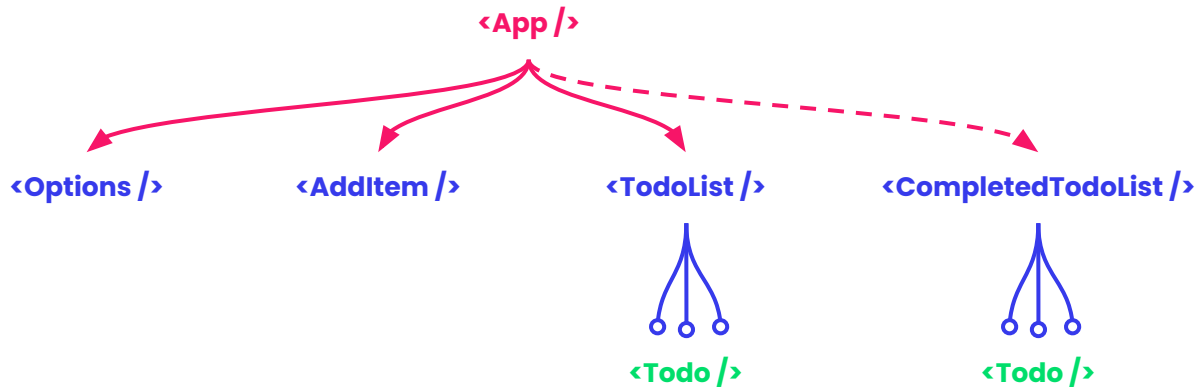
```
<ul className="completed-list">
  {completedTodos.map(todo => (
    <Todo
      key={todo.text}
      todo={todo}
      removeTodo={props.removeTodo}
      toggleTodoCompletion={props.toggleTodoCompletion}
    />
  ))}
</ul>
```

Notice the key prop being passed to our Todo component. Whenever you create **multiple instances** of the same component, React needs you to give them a **unique key**, so it can manage them more efficiently

Q: How is this key unique?



State tree

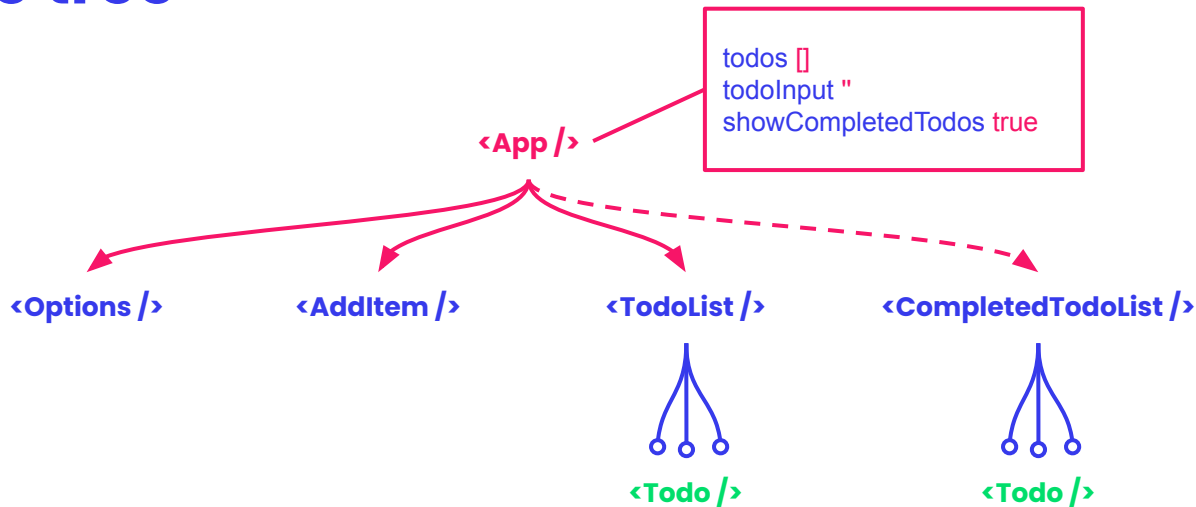


Now, when thinking about our **app state**, it's important to start mapping in our heads where each state is, since **any component** can potentially have state

Q: What state do we currently have? Where is it?



State tree

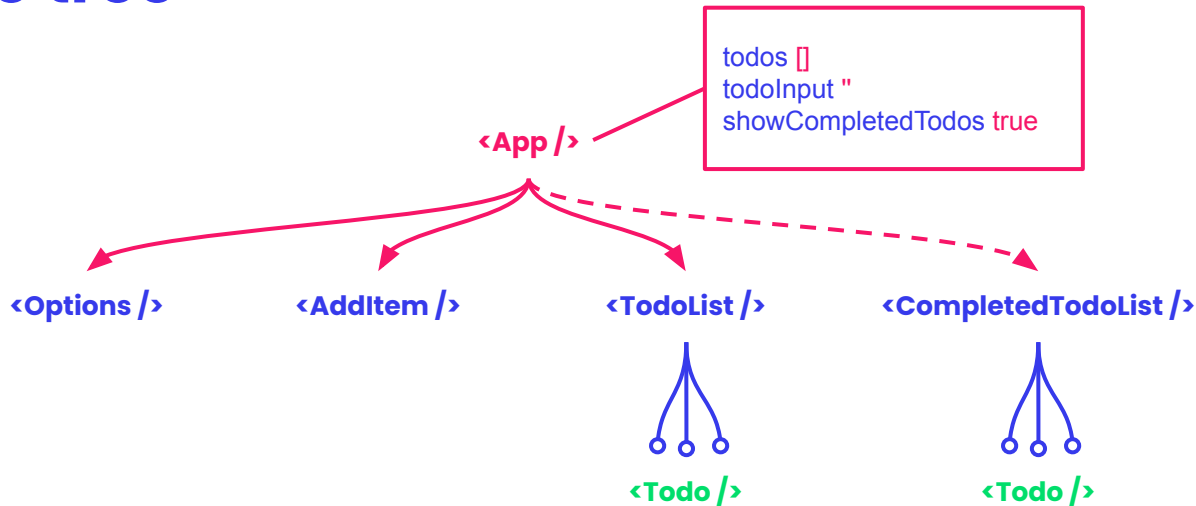


A: That's right, currently all of our state is in the **App** component

On such a small app, this is not a problem, but as your application grows, you **don't want** all of your state to be at the **top**, because you will end up with a **giant app component**, and components are easy when they are small



State tree

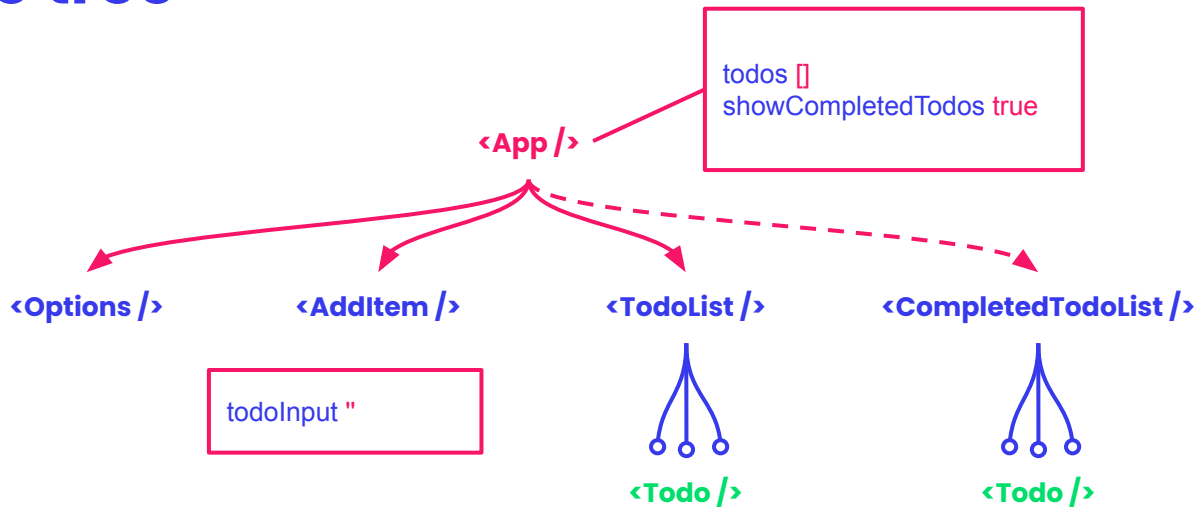


To solve this, we can try to keep our state as **low** down the tree as possible – REMEMBER as a general rule, keep state local, keep state minimal

Q: Can we move any state down the tree? Which one? Why?



State tree



A: Yup, `todoInput` is only needed by `AddItem`, so we can move it down

We will now need to make some more changes to make sure everything still works



Time for some live coding

Let's move that state together on this app!

Here's a finished version



Time for some practice

<https://github.com/boolean-uk/react-props-practice-2>

Your job is to relocate any state you can as low as possible